

Chuleta de Complejidad: Complejidad, Problemas NP-Completos y FNP

0. Problemas de Complejidad Básica

Reconocimiento de Palíndromos — en L

Formalización: $PAL(x)$.

Enunciado: Dada una palabra x de longitud n , decidir si es un palíndromo (x se lee igual de izquierda a derecha que de derecha a izquierda).

Clase: L ($\subseteq$ P). **Tiempo:** $O(n^2)$. **Espacio:** $O(\log n)$.

Idea: La clave es que para comprobar el par de posiciones $(i, n+1-i)$ solo hace falta recordar el valor de i (un contador de $\lceil \log n \rceil$ bits) y el carácter leído en la posición i (una constante). El algoritmo recorre $i = 1, \dots, \lfloor n/2 \rfloor$: para cada i , avanza hasta la posición i en la entrada y memoriza ese carácter; luego busca la posición simétrica $n+1-i$ avanzando desde el otro extremo y compara. Si algún par difiere, rechaza; si todos coinciden, acepta. El estado de trabajo es únicamente el índice i y un símbolo — nada más.

Existencia de Caminos (CAMINO) — NL-completo

Formalización: $CAMINO(G, s, t)$, con $G = (V, E)$. Para todo $\Pi \in NL$, $\Pi \propto CAMINO(G, s, t)$.

Enunciado: Dado un grafo dirigido $G = (V, E)$ con $v = |V|$ nodos y dos nodos s, t , decidir si existe un camino de s a t .

Clase: NL-completo.

Por qué NL-completo: Todo cómputo no determinista en espacio $O(\log n)$ produce un grafo de configuraciones de tamaño polinómico en el que cada configuración es un nodo y cada paso de cálculo es un arco. La máquina acepta si y solo si existe camino desde la configuración inicial hasta alguna de aceptación — exactamente el problema CAMINO. Por tanto cualquier problema de NL se reduce a CAMINO.

Tres algoritmos, tres compromisos tiempo/espacio:

- **Determinista (BFS/DFS).** Tiempo $O(n^2)$, Espacio $O(n)$. Se guardan dos listas: nodos visitados y nodos pendientes, ambas de longitud $\leq v$. Por cada nodo extraído de la lista se recorre la lista de arcos en $O(n)$ pasos, y hay a lo sumo $v \leq n$ nodos: total $O(n^2)$ tiempo y $O(n)$ espacio (las dos listas).
- **No determinista.** Tiempo $O(v^2)$, Espacio $O(\log v)$. Se mantiene solo el nodo actual (un nombre de $\lceil \log v \rceil$ bits). En cada paso se adivina un vecino y se sobrescribe el nodo actual con él, avanzando hasta v pasos. Si en algún momento el nodo actual es t , acepta. El espacio de trabajo es un único nombre de nodo.
- **Savitch (determinista, espacio mínimo).** Tiempo $O(v^{2 \log v})$, Espacio $O(\log^2 v)$. La idea es dividir el problema: “¿hay camino de x a y de longitud $\leq 2^k$?” se resuelve buscando un nodo intermedio z tal que haya camino de x a z y de z a y , ambos de longitud $\leq 2^{k-1}$. La recursión tiene profundidad $\lceil \log v \rceil$ y en cada nivel solo se almacenan tres nodos (x, y, z) y el contador k , es decir $O(\log v)$ bits por nivel, totalizando $O(\log^2 v)$.

Flujo Máximo — en P

Formalización: $FM(G, c, s, t)$.

Enunciado: Dado un grafo dirigido con capacidades $c : E \rightarrow \mathbb{N}$, un origen s y un destino t , encontrar el flujo de valor máximo de s a t (sin superar ninguna capacidad y conservando el flujo en cada nodo intermedio).

Clase: P. **Tiempo:** $O(m^3)$ con $m = |V|$, equivalentemente $O(n^5)$ en la longitud de la entrada. **Espacio:** $O(n)$.

Idea (Ford-Fulkerson): En cada iteración se busca un *camino aumentante*: un camino de s a t en el que cada arco (u, v) tenga capacidad residual positiva, es decir, que pueda recibir más flujo ($c(u, v) - f(u, v) > 0$) o que el flujo actual pueda cancelarse ($f(v, u) > 0$, modelado como un arco inverso). Se aumenta el flujo en la cantidad mínima disponible a lo largo del camino y se repite. Cuando no hay más caminos aumentantes, el flujo es máximo.

Por qué BFS garantiza tiempo polinómico: si siempre se elige el camino más corto (BFS), la longitud mínima de los caminos aumentantes nunca decrece. Esto implica que cada arco solo puede ser “cuello de botella” a lo sumo $O(m)$ veces antes de que todos los caminos por él se alarguen irreversiblemente. Con $O(m)$ arcos y $O(m)$ veces cada uno, el total de aumentos es $O(m^2)$; como cada BFS cuesta $O(m)$, el tiempo total es $O(m^3)$.

Sin BFS puede ser exponencial: con capacidades de magnitud C , el algoritmo puede elegir en cada paso el mismo camino corto mejorando solo una unidad, acumulando $2C$ iteraciones — exponencial en $\log C$ (el número de bits necesario para escribir C).

Parejas (PAR) — en P

Formalización: $PAR(V_1, V_2, A) \propto FM(G, c, s, t)$, con $|V_1| = |V_2| = q$ y $\exists A' \subseteq A / \forall v_1 \in V_1, \exists! v_2 \in V_2$ tal que $(v_1, v_2) \in A'$.

Enunciado: Dados conjuntos V_1, V_2 del mismo tamaño q y compatibilidades $A \subseteq V_1 \times V_2$, decidir si existe un emparejamiento perfecto (cada elemento aparece en exactamente un par).

Clase: P. **Tiempo:** $O(q^3)$ (heredado del Flujo Máximo). **Espacio:** $O(n)$.

Idea de la reducción a Flujo Máximo: Se construye una red de flujo: fuente s conectada a cada nodo de V_1 ; arcos de V_1 a V_2 según A ; cada nodo de V_2 conectado al sumidero t ; todas las capacidades iguales a 1. La capacidad 1 garantiza que cada nodo solo puede ser emparejado con uno del otro lado. Un flujo de valor q implica que los q nodos de V_1 envían cada uno exactamente 1 unidad a un nodo distinto de V_2 , lo que corresponde biyectivamente a un emparejamiento perfecto.

1. Problemas de Lógica Proposicional

Consistencia en Lógica Proposicional (SAT)

Formalización: $SAT(U, C) \equiv \exists \varphi : U \rightarrow \{V, F\} / C_1 \wedge \dots \wedge C_h \equiv V$, con cada $C_i \in C$ disyunción de literales. Para todo $\Pi \in NP$, $\Pi \propto SAT(U, C)$.

Enunciado: Un conjunto $U = \{p_1, \dots, p_m\}$ de símbolos proposicionales y una colección C de cláusulas sobre estos símbolos. PREGUNTA: ¿Son consistentes las cláusulas?

En NP: Un algoritmo no determinista selecciona un valor de verdad (V o F) para cada símbolo en U y verifica en tiempo polinómico si cada cláusula queda satisfecha.

NP-completo (Cook-Levine): Todo problema en NP se reduce a SAT.

Idea de la reducción: Dado $\Pi \in NP$, existe una MT no determinista M que lo decide en tiempo polinómico $p(n)$. Se introducen cuatro tipos de variables booleanas: $Q[i, k]$ (la máquina está en el estado q_k en el instante i), $H[i, j]$ (la cabeza está en la casilla j), $S[i, j, k]$ (la casilla j contiene el símbolo s_k) y $O[i, k]$ (se elige la opción k). Con ellas se construyen grupos de cláusulas que codifican las leyes físicas del cómputo: unicidad de estado, posición y contenido en cada instante; transiciones válidas según δ ; configuración inicial fijada por la entrada; y la condición de aceptación $Q[p(n), 1]$. La fórmula es consistente si y solo si M acepta la entrada, y tiene tamaño polinómico en n .

3-SAT exacto

Formalización: $SAT(U, C) \propto 3\text{-SAT}(U', C')$.

Enunciado: Un conjunto de variables U y un conjunto de cláusulas C en las que cada cláusula tiene exactamente 3 literales (pueden repetirse variables). PREGUNTA: ¿Existe una asignación de valores de verdad tal que en cada cláusula haya al menos un literal verdadero?

En NP: Es un caso particular de SAT; el mismo algoritmo no determinista de adivinación y verificación es válido.

Se reduce desde: SAT.

Idea de la reducción: Se usa la equivalencia: la cláusula $P \vee Q$ es satisfacible si y solo si $(P \vee x) \wedge (\neg x \vee Q)$ lo es para una nueva variable x . Cada cláusula de SAT con $k \geq 4$ literales $l_1 \vee \dots \vee l_k$ se fragmenta iterativamente $k - 3$ veces introduciendo variables auxiliares $x_2, \dots, x_{k-2}$, generando la cadena: $(l_1 \vee l_2 \vee x_2), (\neg x_2 \vee l_3 \vee x_3), \dots, (\neg x_{k-2} \vee l_{k-1} \vee l_k)$. Este conjunto de cláusulas de tamaño 3 es satisfacible si y solo si lo es la cláusula original.

3-SAT restringido

Formalización: $3\text{-SAT}(U, C) \propto 3\text{-SAT}_R(U', C')$.

Enunciado: Un conjunto de variables U y un conjunto de cláusulas C en las que cada cláusula tiene exactamente 3 literales *distintos* (ninguna variable se repite dentro de una cláusula, cada literal aparece a lo más dos veces en toda la fórmula y cada variable a lo más tres veces). PREGUNTA: ¿Existe una asignación satisfaciente?

En NP: Caso particular de 3-SAT exacto; el mismo algoritmo no determinista es válido.

Se reduce desde: 3-SAT exacto.

Idea de la reducción: Se añaden tres variables auxiliares x_1, x_2, x_3 forzadas a ser falsas: se incluyen todas las cláusulas de tamaño 3 sobre $\{x_1, x_2, x_3\}$ excepto $(x_1 \vee x_2 \vee x_3)$. Estas 7 cláusulas se satisfacen *únicamente* con $x_1 = x_2 = x_3 = F$. A continuación, cualquier cláusula original con menos de 3 literales distintos se rellena

añadiendo x_1 o x_2 según sea necesario, reproduciendo la semántica original (pues esas variables siempre son falsas).

2-SAT — en $NL \subseteq P$, *no* NP-completo

Formalización: $2SAT(U, C)$.

Enunciado: Un conjunto de variables U y un conjunto de cláusulas C en las que cada cláusula tiene exactamente 2 literales. PREGUNTA: ¿Son consistentes las cláusulas?

En NL ($\subseteq P$): La clave es construir el *grafo de implicaciones*: los nodos son todos los literales posibles (x y $\neg x$ por cada variable); los arcos representan implicaciones derivadas de las cláusulas (la cláusula $l_1 \vee l_2$ equivale a $\neg l_1 \Rightarrow l_2$ y $\neg l_2 \Rightarrow l_1$, por lo que se añaden ambos arcos).

Algoritmo polinómico: El conjunto de cláusulas C es consistente si y solo si no existe ninguna variable x tal que haya un camino de x a $\neg x$ y otro de $\neg x$ a x en el grafo de implicaciones. Comprobar la existencia de caminos en un grafo dirigido es un problema de alcanzabilidad, que pertenece a NL. Como $NL = CoNL$, el complementario (la inconsistencia) también está en NL, luego $2-SAT \in NL \subseteq P$.

NAESAT

Formalización: $3-SAT(U, C) \propto NAESAT(U', C')$.

Enunciado: Un conjunto de variables U y un conjunto de cláusulas C de exactamente 3 literales. PREGUNTA: ¿Existe una asignación tal que en cada cláusula haya al menos un literal verdadero y al menos uno falso?

En NP: Se asignan valores de forma no determinista y se comprueba en tiempo polinómico que en cada cláusula no todos los literales tienen el mismo valor de verdad.

Se reduce desde: 3-SAT.

Idea de la reducción: Se añade una variable global z . Para cada cláusula $(p \vee q \vee r)$ se introduce una variable puente s y se sustituye la cláusula por cuatro: $(s \vee r \vee z)$, $(p \vee q \vee \neg s)$, $(\neg p \vee s \vee z)$, $(\neg q \vee s \vee z)$. La clave es la simetría de NAESAT: si $(x_1, \dots, x_n)$ es solución, también lo es $(\neg x_1, \dots, \neg x_n)$. Por tanto, siempre puede elegirse una solución con $z = F$; en ese caso las cuatro cláusulas colapsan a la condición de que $p \vee q \vee r$ sea satisfecha, recuperando la instancia original de 3-SAT.

MAX2SAT

Formalización: $3-SAT(U, C) \propto MAX2SAT(C', K)$.

Enunciado: Un conjunto de cláusulas con exactamente dos literales y un entero $K \geq 0$. PREGUNTA: ¿Pueden satisfacerse al menos K cláusulas?

En NP: Se adivina una asignación y se cuenta el número de cláusulas satisfechas, verificando en tiempo polinómico que es $\geq K$.

Se reduce desde: 3-SAT.

Idea de la reducción: Técnica de gadgets. Cada cláusula $(x \vee y \vee z)$ se traduce en 10 cláusulas de 2 literales introduciendo una variable nueva w : (x) , (y) , (z) , (w) , $(\neg x \vee \neg y)$, $(\neg y \vee \neg z)$, $(\neg z \vee \neg x)$, $(x \vee \neg w)$, $(y \vee \neg w)$, $(z \vee \neg w)$. Si al menos uno de x, y, z es verdadero, se puede elegir w de modo que exactamente 7 de las 10 cláusulas queden satisfechas (pero no más). Si los tres son falsos, a lo sumo 6 se satisfacen, independientemente de w . Fijando $K = 7m$ (donde m es el número de cláusulas originales), el sistema de MAX2SAT tiene solución si y solo si el 3-SAT original la tiene.

Horn-SAT — en P , *no* NP-completo

Formalización: $HORNSAT(U, C)$.

Enunciado: Un conjunto de variables U y un conjunto de cláusulas de Horn (cada cláusula tiene a lo sumo un literal positivo). PREGUNTA: ¿Son consistentes las cláusulas?

En P: Se resuelve en tiempo lineal mediante *propagación unitaria* (greedy). Tres tipos de cláusulas: *hechos* (y) — fuerzan $y = V$; *reglas* $(\neg x_1 \vee \dots \vee \neg x_k \vee y)$ — equivalen a $x_1 \wedge \dots \wedge x_k \Rightarrow y$; *metas* $(\neg x_1 \vee \dots \vee \neg x_k)$ — equivalen a $\neg(x_1 \wedge \dots \wedge x_k)$.

Algoritmo: (1) Poner a V todas las variables que aparecen como hechos. (2) Para cada regla cuyo antecedente sea completamente verdadero, poner a V el consecuente. (3) Repetir hasta que no haya cambios. (4) Poner a F el resto. (5) Comprobar las metas: si alguna queda violada, **INSAT**; en caso contrario, **SAT**. La estrategia greedy (forzar V solo cuando es obligatorio, F por defecto) es óptima porque las implicaciones solo imponen obligaciones en la dirección V .

2. Problemas de Conjuntos

Acoplamiento por tripletas (ACTRI)

Formalización: $3-SAT(U, C) \propto ACTRI(W, X, Y, M)$, con $|W| = |X| = |Y| = q$ y $M \subseteq W \times X \times Y$.

Enunciado: Tres conjuntos disjuntos W, X, Y del mismo tamaño q y un subconjunto $M \subseteq W \times X \times Y$ de compatibilidades. PREGUNTA: ¿Contiene M un subconjunto $M' \subseteq M$ con q elementos tal que para cualesquiera dos tripletas distintas de M' sus componentes en W , en X y en Y son distintas?

En NP: Se elige de forma no determinista un subconjunto de M y se comprueba en tiempo polinómico que tiene q elementos y que cada elemento de W, X e Y aparece exactamente una vez.

Se reduce desde: 3-SAT.

Idea de la reducción: Sea una instancia de 3-SAT con n variables $u_1, \dots, u_n$ y m cláusulas $c_1, \dots, c_m$. Se construyen tres conjuntos W, X, Y :

- **Tripletas de variable (polígono).** Para cada u_i se añaden a W los $2m$ elementos $u_i[1], \dots, u_i[m]$ (forma positiva) y $\bar{u}_i[1], \dots, \bar{u}_i[m]$ (forma negativa); a X los elementos $a_i[1], \dots, a_i[m]$; a Y los elementos $b_i[1], \dots, b_i[m]$. Se definen dos familias de m tripletas: la pista *verdadera* usa $u_i[j]$ (con $a_i[j]$ y $b_i[j]$ correspondientes) y la pista *falsa* usa $\bar{u}_i[j]$ (con los mismos a_i, b_i pero con un desfase cíclico). Los $a_i[j]$ y $b_i[j]$ solo aparecen en estas tripletas, por lo que en cualquier acoplamiento perfecto *se debe elegir exactamente una de las dos pistas*. Elegir la pista verdadera consume todos los $u_i[j]$ y deja libres los $\bar{u}_i[j]$; elegir la falsa hace lo contrario. Esta elección binaria codifica el valor de verdad de u_i .
- **Tripletas de cláusula.** Para cada cláusula c_j se añaden un elemento $s[j]$ a X y $t[j]$ a Y . Por cada literal de c_j se añade la tripleta formada por el elemento de W correspondiente a ese literal en la cláusula j junto con $s[j]$ y $t[j]$. El par $(s[j], t[j])$ debe cubrirse por exactamente una tripleta; esta solo puede elegirse si el elemento de W que usa está libre, es decir, si el literal correspondiente es verdadero. Así, el acoplamiento puede completar la cláusula j si y solo si al menos un literal de c_j está satisfecho.
- **Tripletas de relleno.** Tras cubrir las cláusulas quedan libres $m(n-1)$ elementos de W . Se añaden $m(n-1)$ pares nuevos $(g[k], h[k])$ a X e Y , compatibles con todos los elementos sobrantes de W , para que el acoplamiento pueda completarse sea cual sea la asignación elegida.

Cubrimiento por conjuntos de tamaño tres (3-SET)

Formalización: $ACTRI(W, X, Y, M) \propto 3\text{-SET}(X, C)$, con $|X| = 3q$.

Enunciado: Un conjunto finito X con $|X| = 3q$ y una familia C de subconjuntos de X de exactamente tres elementos. PREGUNTA: ¿Existe $C' \subseteq C$ tal que $X = \bigcup_{A \in C'} A$ y los elementos de C' son disjuntos dos a dos?

En NP: Se elige C' de forma no determinista y se verifica en tiempo polinómico que su unión recubre X y sus miembros son disjuntos.

Se reduce desde: ACTRI.

Idea de la reducción: Transformación directa. Dado un ejemplo de ACTRI con conjuntos W, X, Y y tripletas M , se define $X_{\text{nuevo}} = W \cup X \cup Y$ y $C = \{\{w, x, y\} : (w, x, y) \in M\}$. Todo acoplamiento perfecto de ACTRI (que cubre cada elemento exactamente una vez) corresponde biyectivamente a una cubierta exacta de 3-SET, y viceversa.

Conjunto Mínimo de Tests (CMT)

Formalización: $ACTRI(W, X, Y, M) \propto CMT(A, C, J)$.

Enunciado: Un conjunto finito A (posibles diagnósticos), una familia C de subconjuntos de A (posibles tests) y un entero $J \in \mathbb{N}$. PREGUNTA: ¿Existe una subfamilia $C' \subseteq C$ con $|C'| \leq J$ tal que para cada par de elementos distintos $a_i, a_j \in A$ exista algún test $c \in C'$ que contenga exactamente uno de ellos?

En NP: Se elige C' de forma no determinista y se comprueba en tiempo polinómico que distingue todo par de diagnósticos.

Se reduce desde: ACTRI.

Idea de la reducción: Reemplazamiento local con refuerzo. Dados W, X, Y con $|W| = |X| = |Y| = q$ y tripletas M , se construye $A = W \cup X \cup Y \cup \{w_0, x_0, y_0\}$, los tests son $\{\{w, x, y\} : (w, x, y) \in M\}$ más dos tests globales $W \cup \{w_0\}$ y $X \cup \{x_0\}$, y se fija $J = q + 2$. Los dos tests globales son los únicos que distinguen w_0 de los elementos de W (y x_0 de los de X), por lo que son obligatorios. Esto consume el presupuesto de 2 tests, dejando exactamente q tests para distinguir el resto; esos q tests forman una partición perfecta de $W \cup X \cup Y$ si y solo si ACTRI tiene solución.

3. Problemas de Grafos

Cubrimiento por vértices (CV)

Formalización: $3\text{-SAT}(U, C) \propto CV(G = (V, E), K)$.

Enunciado: Dado un grafo $G = (V, E)$ y un número natural $K \leq |V|$, determinar si existe $C \subseteq V$ de tamaño $\leq K$ tal que toda arista de G tiene al menos un extremo en C .

En NP: Se escogen $\leq K$ vértices de forma no determinista y se verifica en tiempo polinómico que toda arista queda cubierta.

Se reduce desde: 3-SAT.

Idea de la reducción: Para cada variable u_i se añaden dos vértices u_i y $\bar{u}_i$ unidos por una arista. Para cada cláusula c_j se añade un triángulo $(a_1[j], a_2[j], a_3[j])$ cuyos tres vértices se conectan con los vértices-variable de sus respectivos literales. El límite $K = n + 2m$, donde n = número de variables y m = número de cláusulas, es ajustado: para cubrir todas las aristas hacen falta al menos n vértices (uno por cada par de variable) y al menos $2m$ (dos por cada triángulo). La restricción fuerza exactamente un vértice por par $(u_i, \bar{u}_i)$, determinando el valor de verdad de cada variable, y exactamente dos por triángulo. El tercer vértice del triángulo que queda sin cubrir tiene su única arista hacia el grafo cubierta por el vértice-variable correspondiente, cuyo valor de verdad satisface esa cláusula.

Clique máximo (CM)

Formalización: $CV(G, K) \propto CM(\bar{G}, J = |V| - K)$.

Enunciado: Dado un grafo $G = (V, E)$ y un número natural $J \leq |V|$, determinar si existe un subconjunto $B \subseteq V$ de tamaño $\geq J$ tal que $(v_1, v_2) \in E$ para todo $v_1, v_2 \in B$.

En NP: Se elige un subconjunto de nodos de tamaño $\geq J$ de forma no determinista y se verifica en tiempo polinómico que toda pareja figura como arista de E .

Se reduce desde: Conjunto Independiente (CI) / Cubrimiento por Vértices (CV).

Idea de la reducción: Los tres problemas son equivalentes. $B \subseteq V$ es un clique en $\bar{G}$ (con aristas $\bar{E} = V \times V \setminus E$) si y solo si B es un conjunto independiente en G . A su vez, B es conjunto independiente en G si y solo si $V \setminus B$ es cubrimiento por vértices de G . Así, un clique de tamaño $\geq n - K$ en $\bar{G}$ equivale directamente a un cubrimiento por vértices de tamaño $\leq K$ en G .

Conjunto Independiente (CI)

Formalización: $CV(G = (V, E), K) \propto CI(G = (V, E), J = n - K)$.

Enunciado: Dado un grafo $G = (V, E)$ y un número natural $J \leq |V|$, determinar si existe $A \subseteq V$ de tamaño $\geq J$ tal que $(u, v) \notin E$ para todo $u, v \in A$.

En NP: Se elige un subconjunto de nodos de forma no determinista y se verifica en tiempo polinómico que ningún par está conectado.

Se reduce desde: Cubrimiento por Vértices (CV).

Idea de la reducción: Equivalencia directa. Si V^* es un cubrimiento por vértices de tamaño $\leq K$, entonces $V \setminus V^*$ es un conjunto independiente de tamaño $\geq n - K$: toda arista tiene al menos un extremo en V^* , por tanto sus dos extremos no pueden pertenecer ambos a $V \setminus V^*$.

Circuito Hamiltoniano (CH)

Formalización: $CV(G, K) \propto CH(G')$.

Enunciado: Dado un grafo no dirigido $G = (V, E)$, determinar si existe una sucesión $(v_1, \dots, v_n)$ tal que cada nodo de V aparece exactamente una vez, existe una arista entre v_i y v_{i+1} para $1 \leq i < n$, y existe una arista entre v_n y v_1 .

En NP: Se genera una permutación de n nodos de forma no determinista y se verifica en tiempo polinómico que los arcos consecutivos y el de cierre pertenecen a E .

Se reduce desde: Cubrimiento por Vértices (CV).

Idea de la reducción: Diseño de componentes. Cada arista $e = (u, v)$ de G se sustituye por un gadget de 12 vértices que el circuito debe atravesar obligatoriamente. El gadget admite exactamente dos recorridos: de la parte izquierda (lado u) a la derecha (lado v), o al revés. Los gadgets de un mismo vértice v se encadenan en secuencia. Se añaden K vértices selectores $a_1, \dots, a_K$, cada uno conectado al inicio y al final de la cadena de gadgets de algún vértice. El circuito hamiltoniano, para poder recorrer todos los gadgets, debe entrar en la cadena de cada vértice del cubrimiento desde un selector a_i , recorrerla íntegramente y volver al siguiente selector; la elección de por qué vértice se entra en cada gadget reproduce exactamente la selección de los K vértices del cubrimiento.

Viajante de Comercio (VC) — Versión decisión

Formalización: $CH(G) \propto VC(C = V, d, B = m)$.

Enunciado: Un conjunto finito de ciudades $C = \{c_1, \dots, c_m\}$, una función de distancia $d : C \times C \rightarrow \mathbb{N}$ y una cota $B \in \mathbb{N}$. PREGUNTA: ¿Existe un circuito que visite todas las ciudades exactamente una vez y tenga coste total $\leq B$?

En NP: Se genera una permutación de ciudades de forma no determinista y se suman las distancias, verificando en tiempo polinómico que no superan B .

Se reduce desde: Circuito Hamiltoniano (CH).

Idea de la reducción: Se construye el grafo completo de ciudades $C = V$. Las aristas presentes en G reciben distancia 1 y las ausentes reciben distancia 2. Con cota $B = m$, la única forma de no superar el presupuesto es recorrer exclusivamente aristas de coste 1, es decir, las aristas originales de G . Cualquier tour de coste $\leq m$ es necesariamente un circuito hamiltoniano de G , y viceversa.

Partición en Triángulos (PARTRI)

Formalización: $3\text{-SET}(X, C) \propto \text{PARTRI}(G, q)$.

Enunciado: Dado un grafo $G = (V, E)$ con $|V| = 3q$, determinar si existe una partición de V en q subconjuntos disjuntos de tamaño 3, $V_1, \dots, V_q$, tal que si $V_i = \{V_{i[1]}, V_{i[2]}, V_{i[3]}\}$ entonces los arcos $(V_{i[1]}, V_{i[2]})$, $(V_{i[2]}, V_{i[3]})$, $(V_{i[1]}, V_{i[3]}) \in E$ (cada subconjunto forma un triángulo).

En NP: Se elige la partición de forma no determinista y se verifica en tiempo polinómico que cada subconjunto induce un triángulo completo.

Se reduce desde: 3-SET.

Idea de la reducción: Reemplazamiento local. Cada elemento de X se convierte en un vértice. Para cada subconjunto $c_i = \{x_i, y_i, z_i\} \in C$ se añaden 9 vértices auxiliares $a_i[1], \dots, a_i[9]$ formando un subgrafo específico conectado a x_i, y_i y z_i . Este subgrafo admite exactamente dos formas de particionarse en triángulos: una que consume los vértices base $\{x_i, y_i, z_i\}$ (correspondiente a incluir c_i en la cubierta de 3-SET) y otra que los ignora y cierra el subgrafo internamente (correspondiente a no incluirlo). La equivalencia entre cubiertas exactas de 3-SET y particiones en triángulos es biyectiva.

Isomorfismo de Grafos (GI) — en NP, estado desconocido

Formalización: $GI(G_1, G_2)$; no se conoce reducción desde/hacia un problema NP-completo.

Enunciado: Dados dos grafos no dirigidos $G_1 = (V_1, E_1)$ y $G_2 = (V_2, E_2)$, determinar si existe un isomorfismo entre ellos, es decir, una aplicación biyectiva $f : V_1 \rightarrow V_2$ tal que $(u, v) \in E_1 \Leftrightarrow (f(u), f(v)) \in E_2$.

En NP: Se adivina de forma no determinista la biyección f y se verifica en tiempo polinómico que preserva todas las aristas (y sus ausencias).

Estado de complejidad: No se ha demostrado que sea NP-completo ni que esté en P. Se conjetura que ocupa un lugar intermedio estricto entre ambas clases. Se define la clase GI como el conjunto de todos los problemas que se pueden reducir al isomorfismo de grafos; GI es GI-completo por definición.

4. Problemas Numéricos y de Partición

SUMA

Formalización: $\text{ACTRI}(W, X, Y, M) \propto \text{SUMA}(A, s, B)$.

Enunciado: Un conjunto A , una función de tamaño $s : A \rightarrow \mathbb{N}$ y un número entero B . PREGUNTA: ¿Existe $A' \subseteq A$ tal que $\sum_{a \in A'} s(a) = B$?

En NP: Se elige el subconjunto de forma no determinista y se suma en tiempo polinómico, verificando la igualdad con B .

Se reduce desde: ACTRI.

Idea de la reducción: A cada tripleta $m_i = (w_{f(i)}, x_{g(i)}, y_{h(i)})$ se le asigna un entero cuya representación binaria se divide en $3q$ zonas de $p = \lfloor \log_2 q \rfloor + 1$ bits, con un 1 al final de las zonas correspondientes a $w_{f(i)}$, $x_{g(i)}$ e $y_{h(i)}$ y 0 en el resto. El ancho p garantiza que al sumar $\leq q$ de estos enteros no se producen acarreos entre zonas. La meta B tiene un 1 al final de cada zona; alcanzarla exactamente obliga a que cada zona acumule exactamente 1, es decir, que cada elemento de W, X e Y aparezca en exactamente una tripleta seleccionada, lo que es exactamente un acoplamiento perfecto de ACTRI.

PARTICIÓN

Formalización: $\text{SUMA}(A, s, B) \propto \text{PARTICIÓN}(C, s')$.

Enunciado: Un conjunto C y una función de tamaño $s : C \rightarrow \mathbb{N}$. PREGUNTA: ¿Existe $C' \subseteq C$ tal que $\sum_{a \in C'} s(a) = \sum_{a \in C \setminus C'} s(a)$?

En NP: Se divide el conjunto en dos partes de forma no determinista y se verifica en tiempo polinómico que sus sumas son iguales.

Se reduce desde: SUMA.

Idea de la reducción: A partir de una instancia de SUMA con conjunto A , tamaños s y objetivo B , se añaden dos elementos balancín $b_1, b_2 \notin A$ con $s(b_1) = \sum_{a \in A} s(a)$ y $s(b_2) = 2B$, y se mantiene s igual para los elementos de A . Su gran tamaño impide que caigan en la misma mitad. Al quedar en lados opuestos, la mitad que contiene

b_1 necesita que los elementos ordinarios que la acompañan sumen exactamente B para igualar a la otra mitad (que contiene b_2 de peso $2B$ junto con otros elementos). Esto reproduce exactamente la instancia original de SUMA.

Problema de la Mochila (Decisión)

Formalización: $\text{PARTICIÓN}(C, s) \propto \text{MOCHILA}(U, s, v, B, K)$.

Enunciado: Un conjunto finito de objetos U , con tamaño $s(u) \in \mathbb{N}$ y valor $v(u) \in \mathbb{N}$ por objeto, y dos naturales B (tamaño máximo) y K (valor mínimo). PREGUNTA: ¿Existe $U' \subseteq U$ tal que $\sum_{u \in U'} s(u) \leq B$ y $\sum_{u \in U'} v(u) \geq K$?

En NP: Se elige el subconjunto de forma no determinista y se verifican en tiempo polinómico las condiciones de peso y valor.

Se reduce desde: PARTICIÓN.

Idea de la reducción: Técnica de restricción. PARTICIÓN es el caso particular del problema de la Mochila en que $s(u) = v(u)$ para todo objeto y $B = K = \frac{1}{2} \sum_{u \in U} s(u)$. Estas restricciones colapsan el problema de optimización bidimensional (peso y valor) a la única condición de equilibrar exactamente dos mitades de igual suma.

Asignación en un Multiprocesador

Formalización: $\text{PARTICIÓN}(C, s) \propto \text{MULTIPROC}(A, l, m=2, D)$.

Enunciado: Un conjunto A de tareas con longitud $l(a) \in \mathbb{N}$ por tarea, un número de procesadores m y un tiempo límite $D \in \mathbb{N}$. PREGUNTA: ¿Existe una partición $\{A_1, \dots, A_m\}$ de A en m subconjuntos disjuntos tal que $\max \{\sum_{a \in A_i} l(a) : 1 \leq i \leq m\} \leq D$?

En NP: Se elige la partición de forma no determinista y se comprueba en tiempo polinómico que el tiempo máximo de ningún procesador supera D .

Se reduce desde: PARTICIÓN.

Idea de la reducción: Técnica de restricción. PARTICIÓN es el caso particular con $m = 2$ procesadores y $D = \frac{1}{2} \sum_{a \in A} l(a)$. Minimizar el tiempo máximo de dos procesadores bajo este techo es exactamente pedir una partición equilibrada de las tareas, que es el problema de la PARTICIÓN.

5. Problemas de Funciones (FNP)

FSAT — FNP-completo

Formalización: Para todo $\Pi \in \text{FNP}$, $\Pi \propto \text{FSAT}(U, C)$.

Enunciado: Dado un conjunto U de variables proposicionales y un conjunto de cláusulas C , encontrar una asignación de valores de verdad que satisfaga todas las cláusulas, o devolver ε si no existe ninguna.

Clase: FNP-completo. Todo problema en FNP se reduce a FSAT de la misma manera que todo problema en NP se reduce a SAT.

Nota (FNP = FP $\Leftrightarrow$ NP = P): Si $\text{NP} = \text{P}$, se puede encontrar la asignación en tiempo polinómico: dado un oráculo para SAT, se evalúa cada variable x_i fijándola primero a V (generando C_1) y comprobando si C_1 es consistente. Si lo es, se mantiene $x_i = V$ y se continúa con C_1 ; si no, se fija $x_i = F$ y se continúa con C_2 . Repitiendo para todas las variables se obtiene la asignación completa.

Factorización en primos — en FNPT

Formalización: $\text{FACT}(p)$; sin reducción conocida, se conjetura $\text{FACT} \notin \text{FP}$.

Enunciado: Dado un número natural p , calcular su descomposición en factores primos $p = p_1^{k_1} \cdot p_2^{k_2} \cdot \dots \cdot p_m^{k_m}$ junto con un certificado de primalidad para cada p_i .

Clase: FNPT (FNP total). El problema de decisión asociado —determinar si p es primo— está en NP por el *Teorema de Pratt*: $p > 1$ es primo si y solo si existe $1 < r < p$ tal que $r^{p-1} \equiv 1 \pmod{p}$ y $r^{(p-1)/q} \not\equiv 1 \pmod{p}$ para todo divisor primo q de $p-1$. El certificado es una estructura recursiva: r , la lista completa de divisores primos $q_1, \dots, q_k$ de $p-1$, y certificados de primalidad para cada q_i (excepto 2, que no necesita certificado). La longitud total del certificado es $O(\log^2 p)$.

Estado: La factorización siempre existe (FNPT), pero no se conoce algoritmo general polinómico; se conjetura que no está en FP.

Viajante de Comercio (Optimización) — FNP, NP-difícil

Formalización: $\text{VC}_{\text{dec}}(C, d, B) \propto_T \text{VC}_{\text{opt}}(C, d)$.

Enunciado: Dado un conjunto de ciudades C y una función de distancias $d : C \times C \rightarrow \mathbb{N}$, encontrar el circuito que visita todas las ciudades exactamente una vez con coste total mínimo.

Clase: FNP, NP-difícil. Se reduce Turing ($\propto_T$) desde la versión de decisión, que es NP-completa.

Algoritmo con oráculo de decisión: (1) Cota superior $R = (\text{distancia máxima}) \times |C|$. (2) Búsqueda binaria en $[0, R]$ con llamadas al oráculo para determinar el coste óptimo K^* . (3) Para cada arco (c_i, c_j) con coste c : incrementar temporalmente su coste a $c+1$ y preguntar si existe tour de coste $\leq K^*$. Si la respuesta es negativa, el arco es necesario en el óptimo (se restaura el coste original). Si es positiva, el arco no es necesario (se mantiene el coste $c+1$ para excluirlo). Al finalizar, los arcos restaurados forman el circuito óptimo.

Red Feliz — en FNPT

Formalización: $REDFELIZ(V, E, w)$; sin reducción conocida (FNPT).

Enunciado: Dado un grafo (V, E) con pesos $w : E \rightarrow \mathbb{Z}$ (positivos o negativos), encontrar un estado $s : V \rightarrow \{-1, 1\}$ tal que todos los nodos sean felices, es decir, $s(i) \cdot \sum_{(i,j) \in E} s(j) \cdot w(i, j) \geq 0$ para todo $i \in V$.

Clase: FNPT. El problema siempre tiene solución. Se define $\Phi(s) = \sum_{(i,j) \in E} s(i) s(j) w(i, j)$. Si i es infeliz, su contribución a Φ es $C_i = s(i) \cdot \sum_j s(j) w(i, j) = -\delta < 0$. Al invertir $s(i) \rightarrow -s(i)$, solo cambian los términos de i : $C'_i = -C_i = +\delta$, luego $\Phi(s') = \Phi(s) + 2\delta$. Como Φ está acotada ($|\Phi| \leq \sum |w(i, j)|$) y cada inversión la incrementa estrictamente, el proceso termina; al terminar, no queda ningún nodo infeliz. El algoritmo es pseudopolinómico: δ puede ser tan pequeño como 1 y Φ tan grande como $\sum |w(i, j)|$ (exponencial en el tamaño de la entrada en binario).